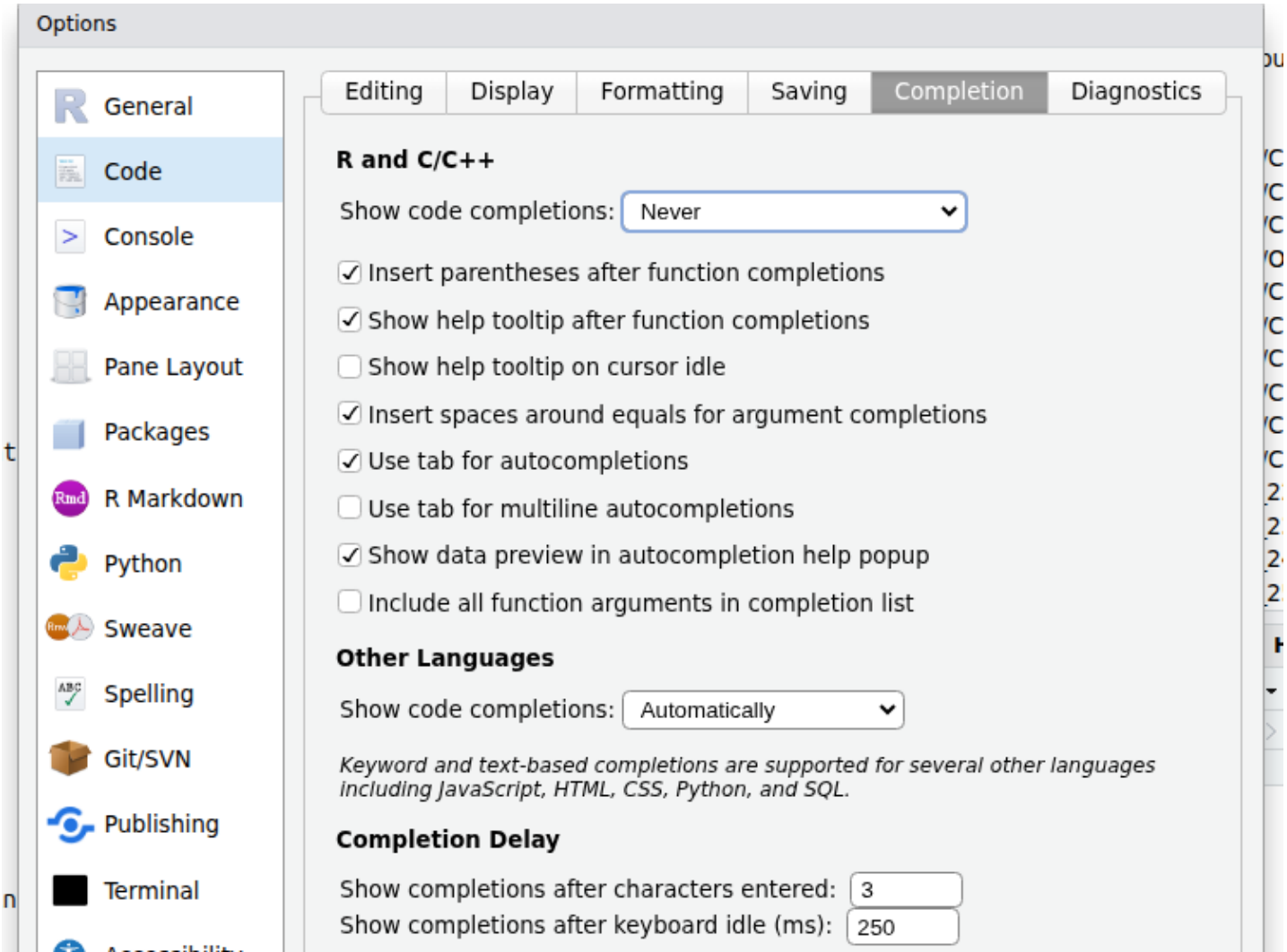
Lab 7: Introduction to R

Matteo Fasiolo

Rstudio set up (ignore if you have already done this)

Rstudio has auto-suggestion features which we want to disable.

To do that go to `Tools` > `Global Options` > `Code` > `Completion` and set `Show code completions` to `Never`.



Pre-sessional work

If you are on university PCs, you can skip the first two steps:

1. Download R
    - https://www.stats.bris.ac.uk/R/
2. Install and Launch RStudio
    - https://docs.posit.co/ide/user/download.html
3. Try code blocks in the slides
    - Write code in the code editor and press ctrl+enter to execute the code line by line.
    - Create R scripts and save them.

Homework 1: prime numbers revisited

Write a program that determines the number of primes smaller or equal than a natural number `n, n>=2`.

Recall:

- A factor is an integer that divides exactly into a whole number without a remainder. E.g., 3 is a factor of 12.
- A prime has only 2 factors, 1 and itself.

Pseudo Code, suppose that `n` is the number of interest:

Loop over i going from 2 to n (1 is not a prime!)

1. Loop over j going from 1 to i
    - If j is factor of i add it to the count of factors of i
2. If i has only two factors, it is a prime and it should be added to the number of primes $<=$ n

You should:

1. Translate above pseudo code into R code

- Use only for() loops.
- Write your code in the code editor and test it.
- After the execution, check the "environment pane" on the top right corner of Rstudio:
    - What are the variables?
    - Why do they have the value they hold?

2. What is the computational complexity of our code?
    - Hint: count how many loop iterations will be executed when the program runs.

3. Time the execution of your code using `Sys.time()`

```
start_time <- Sys.time()
#your code here
end_time <- Sys.time()
end_time - start_time

# Time difference of 0.0009999275 secs
```

Set `n <- 5000` and time the execution (select all and ctrl + enter).

How long does it take?

Predict how long it will take when setting `n <- 10000` before running the code.

Validate your prediction by actually running the code.

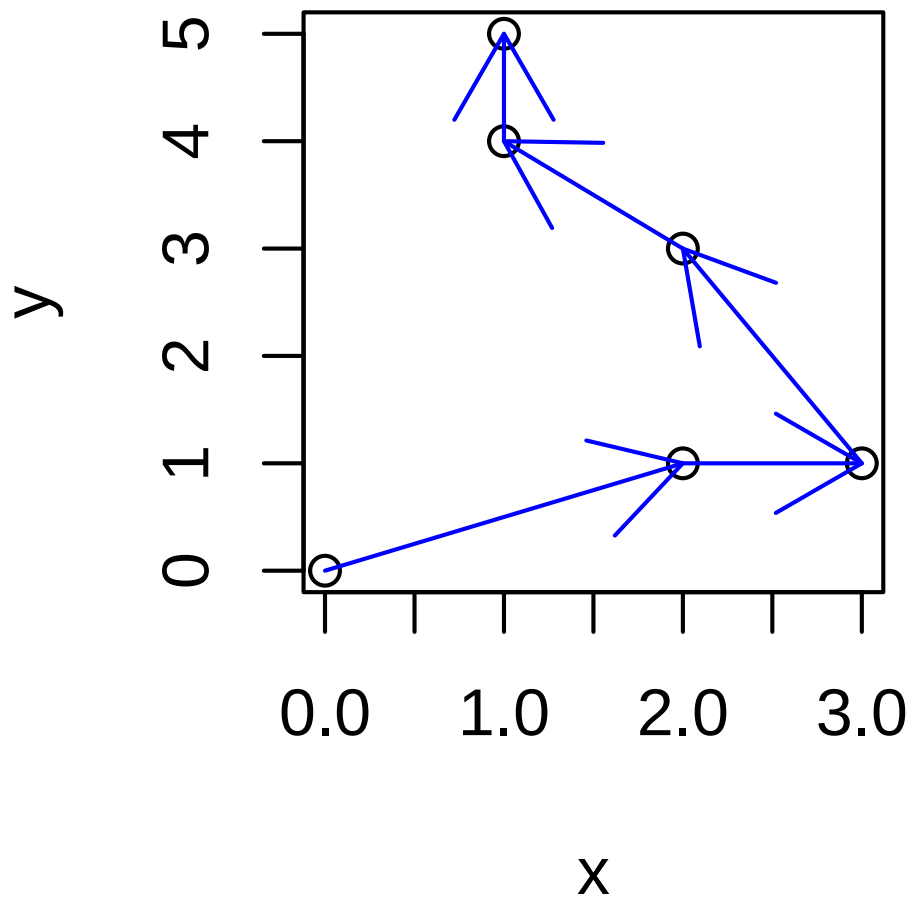
4. Try to wrap your R code into a function.
5. Write the same prime number counting program in C, compile and run it:
    - When setting `n = 5000`, which programming language is faster? faster by how much?
    - Use the skeleton C code in `Lab_Primes_template.c`.

Homework 2

Problem: given a sequence of locations in space:

a. Find the length and direction of the segments separating each pair of locations.

b. Find the total length of the path.



Start by writing a function `dist(a, b)`, where `a` and `b` are vectors represent two points in a space.

`dist` outputs the Euclidean distance between two points.

Requirements:

- Your function should work for input vectors in **any dimension**.
- `dist(v,0)` will return the length of vector `v`.
- **No for-loops**.
- Make test cases, and check your function.

Write an R function `angle`, takes two vectors: `a` and `b`.

It outputs the angle between `a` and `b` in degrees.

- The result should be returned **in degrees**, not in radian.
- Use `dist` function you just wrote.
- Make test cases, and check your function.
- Hint: Type `?acos` in command line and read the documentation.

Consider a sequence of n location in d-dimensional space. E.g.:

```
     x y
loc1 0 0
loc2 2 1
loc3 3 1
loc4 2 3
loc5 1 4
loc6 1 5
```

Write a function that, given a matrix of locations in 2D, returns:

1. a vector containing the length (Euclidean norm) of each step
2. a vector containing the orientation of each step
    - Measure orientation anticlockwise from the positive x-axis, giving a value between 0° and 360°
3. a scalar representing the total length of the path

Hint: to write a function that returns several objects you can use:

```
my_function <- function(){
  a <- 1:2
  b <- "Hello!"
  return( list(a, b) )
}
```

```
x <- my_function()
x[[1]]
```

```
[1] 1 2
```

```
x[[2]]
```

```
[1] "Hello!"
```

That's all you need to know about lists for the moment!

Hint: if you want to check your code visually you can do:

```
locs <- matrix(c(0, 2, 3, 2, 1, 1,
                 0, 1, 1, 3, 4, 5), 6, 2)

plot(locs)

# YOUR CODE
for(ii in list_of_segments){
  # YOUR CODE to compute start (x0, y0) and
  # end (x1, y1) of segments
  arrows(x0, y0, x1, y1)
}
```

Consider the following code:

```
locs_2 <- locs[nrow(locs):1, ]
```

What does this code do?

Try to plot the `locs_2` segments as you did for `locs`, and compare with the two plots.

Does the result make sense?

If not, modify your code to fix the issue.

Challenge: Can you extend your function beyond 2D?

Hint: in $D > 2$ you need more than one angle to determine the direction of a vector!

Submission

Submit your completed `primes.R` and `dist.R` files from Homework 1 and 2.

Before submitting, make sure your code runs.